Random Testing

CMSC 323: Design and Implementation of Programming Languages

This lecture note is adapted from content created by Robby Findler (Northwestern University) and Vincent St-Amour (Northwestern University) and Shriram Krishnamurthi in Programming Languages: Application and Interpretation.

Test Cases

Each test relates a particular input to a particular output

```
(test (bound-ids
        (with ‘x (var 'y) (var 'x)))
        '(x))

(test (binding-ids (with ‘x (var 'y) (var 'x)))
        '(x))
```

Property Testing

But we can only write so many tests by hand.

To find additional bugs, we can automate testing.

We start with what we hope is a fact about our program.

For example,

“if bound-ids says 'x is bound,

Then binding-ids says 'x is binding”

Property Violation

If we can find some WAE for which the property doesn't hold

…

```
(define a-WAE ...)
(bound-ids a-WAE) ; ⇒ '(x)
(binding-ids a-WAE) ; ⇒ '()
```

… we've found a bug.

Property Testing

We can test this property in the usual style.

```
; bound=>binding? : WAE -> boolean
; checks if bound ids are also binding
(define (bound=>binding? e) ...)
(test (bound=>binding? (var 'x))
      #t)
(test (bound=>binding? (with 'x (num 0) (var 'x)))
      #t)
```

Expected result is always true, so if we had lots of WAEs, then we'd have lots of tests.

Property Testing

Or we can write a program to generate test inputs!

Now, I realize that we can use genAI to generate tests.
But as you know, the tests generated may not be accurate.

As an example, I had SpiderAI generate some tests for project 1. One of those tests required that the answer be (positive-leaf 0). But some of you wrote code that gave the alternative correct output of (negative-leaf 0). The AI generated test said that such code failed the test.

Random WAEs

Generating random numbers

```
(random-number : (-> Number))
(define (random-number)
 (case (random 2)
   [(0) 0]
   [(1) (add1 (random-number))]))
```

To use a racket built-in in PLAIT (which doesn't have a built in "random" function), you need to add this line, after #lang plait,

```
(require (typed-in racket/base [random : (Number -> Number)]))
```

Random WAEs

Generate random symbols

```
(random-symbol : (-> Symbol))
(define (random-symbol)
```

Let's assume you want to choose randomly among the following symbols: 'x,'y, 'z, 'a,'b, 'c

Random WAEs

Generate random symbols

```
(random-symbol : (-> Symbol))
(define (random-symbol)
  (local [(define vars (list 'x 'y 'z 'a 'b 'c))]
    (list-ref vars (random (length vars)))))
```

Random WAEs

```
(random-WAE : (Number -> WAE))
(define (random-WAE)
 (case (random 5)
  [(0) (num (random-number))]
```

```
(define-type WAE
  [num (n : Number)]
  [var (id : Symbol)]
  [plus (lhs : WAE)
        (rhs : WAE)]
  [sub (lhs : WAE)
       (rhs : WAE)]
  [with (name : Symbol)
        (named-expression : WAE)
        (body : WAE)])
```

Random WAEs

```
(random-WAE : (Number -> WAE))
(define (random-WAE)
 (case (random 5)
  [(0) (num (random-number))]
  [(1) (var (random-symbol))]
  [(2) (plus (random-WAE)
             (random-WAE))]
  [(3) (sub (random-WAE)
             (random-WAE))]
  [(4) (with (random-symbol)
             (random-WAE)
             (random-WAE))]))
```

Watch out – the code is buggy … (can you see why?)

Generation Strategy

To build a WAE

1/5 of the time, build a number

1/5 of the time, build a symbol

3/5 of the time, first build two more WAEs

Height Bound

Limit WAE size by bounding tree height.

If the height is 0, we choose one of the terminal conditions (base cases) rather than the recursive cases.

```
(random-WAE : (Number -> WAE))
(define (random-WAE h)
 (case (random (if (= h 0) 2 5))
  [(0) (num (random-number))]
  [(1) (var (random-symbol))]
  [(2) (plus (random-WAE (sub1 h))
             (random-WAE (sub1 h)))]
  [(3) (sub (random-WAE (sub1 h))
             (random-WAE (sub1 h)))]
  [(4) (with (random-symbol)
             (random-WAE (sub1 h))
             (random-WAE (sub1 h)))]))
```

Property Implementation

Sublist: let l1 and l2 be lists, l1 is a sublist of another list l2 if all the members of l1 are in l2.

```
(bound=>binding : (WAE -> Boolean))
(define (bound=>binding exp)
  (sublist? (bound-ids exp) (binding-ids exp)))

(sublist? : ((Listof Symbol) (Listof Symbol) -> Boolean))
(define (sublist? l1 l2)
  (cond
    [(empty? l1) #t]
    [(member (first l1) l2) (sublist (rest l1) l2)]
    [else #f]))
```

Property Implementation

Sublist: let l1 and l2 be lists, l1 is a sublist of another list l2 if all the members of l1 are in l2.

```
(bound=>binding : (WAE -> Boolean))
(define (bound=>binding exp)
  (sublist? (bound-ids exp) (binding-ids exp)))

(sublist? : ((Listof Symbol) (Listof Symbol) -> Boolean))
(define (sublist? l1 l2)
  (foldl (lambda (curr-item is-member)
           (and is-member (member curr-item l2))) #t l1))
```

Running Tests

Test bound=>binding with 1000 randomly generated WAEs

```
(test-bound-binding : (Number -> Boolean))
(define (test-bound-binding num-tests)
  (or (= 0 num-tests)
      (and (bound=>binding (random-WAE 3))
           (test-bound-binding (sub1 num-tests)))))

(test-bound-binding 1000)
```

Running Tests

Test bound=>binding with 1000 randomly generated WAEs

(display the tests and test numbers)

```
(define (test-bound-binding num-tests)
 (local [(define awae (random-WAE 3))]
  (begin
   (display (to-string num-tests))
   (display "\n")
   (display (to-string awae))
   (display "\n")
   (or (= 0 num-tests) (and (bound=>binding awae)
                            (test-bound-binding (sub1 num-tests)))))))
```

Exercise

Come up with two other properties that holds between free-ids, binding-ids, bound-ids and shadowed-ids. Write a function that tests these properties on random-WAEs.